Reflective Roadmaps Are Weights: A Training View of Long-Horizon Agent Loops

Anonymous Authors [1]

Abstract

Long-horizon agents fail less like weak reasoners than like systems without durable training state: they forget the objective, repeat local repairs, and restart from brittle transcripts after every interruption. We argue that the missing object is the roadmap. In a persistent agent loop, the roadmap plays the role of weights: a slow, reusable carrier of task structure against which fast local adaptation should be performed. We formalize this view through `Reflective Loop Skills` (`RLS`), a non-parametric training loop for coding agents in which the roadmap is the backbone, `active_task.json` is the adapter, check results are loss signals, local patches are update directions, and `failure_bank.json` is error memory. This framing turns "looping" from a scheduling trick into an optimization interface for long tasks. We instantiate the interface across seven CLI agents and extend it with `moa-loop`, where a single vertical loop becomes a mixture-of-agents computation graph with DAG scheduling and shared blackboard communication. We further separate trigger mechanisms—Stop hooks, external daemons, OS-level cron schedulers with differential health-check intervals, and application-level schedulers—from the persistent state contract they must preserve. The contribution is a training-centric architecture for agents that continue, correct, and accumulate work across epochs without updating neural weights.

1. Introduction

Large language model (LLM) agents have become increasingly capable at tool use, code editing, and autonomous problem solving (Wang et al., 2024b; Novikov et al., 2025; Ou et al., 2025). In software engineering settings, they can inspect repositories, synthesize patches, run tests, and revise their own outputs. Yet the task regime that motivated this work is not the short edit; it is the long task. Long tasks require the agent to keep moving after interruptions, partial failures, context loss, and repeated verification cycles. The dominant interaction pattern remains surprisingly short-lived: a user opens a session, the agent reasons through the current task, local feedback is consumed in the moment, and much of the operational knowledge disappears when the session ends.

Experience-driven agents and procedural memories address parts of this problem by storing trajectories, reflections, workflows, or skills (Zhao et al., 2024; Wang et al., 2024a; 2025; Fang et al., 2025; Han et al., 2025; Xu et al., 2025). These approaches highlight a common insight: capable agents should not rediscover the same behavior every time. Our work arrived at a stronger systems claim: for long tasks, the roadmap becomes as important as the model prompt itself. It is the slow state that preserves what the project is trying to become. Raw transcripts are too noisy for this role, while a single prompt is too brittle to represent both global strategy and local adaptation.

We introduce `Reflective Loop Skills` (`RLS`), a repository-level pattern for persistent, self-correcting coding agents. `RLS` is implemented as seven loop skills targeting common agent CLIs: Codex, Gemini, Claude, Qwen, Kimi, MiniMax, and Cursor, plus `moa-loop` for multi-agent orchestration. Each skill follows the same reflective loop: an optimize pass performs a bounded slice of work, a check pass audits the result, the check pass writes local patches, and the next epoch conditions on those patches. The design deliberately mirrors the deep-learning recipe at the orchestration level. The roadmap is the slow backbone, `active_task.json` is the fast adapter, check findings are loss signals, and local patches are externalized update directions.

Our framing is inspired by machine-learning optimization, but it is explicitly non-parametric. No base model weights are updated. Instead, the system updates external artifacts: roadmaps, task adapters, patches, logs, and failure banks. The analogy is not decorative; it explains why the roadmap matters. Just as pretrained weights carry reusable structure

[1]Anonymous Institution. Correspondence to: Anonymous Authors <anonymous@example.com>.

Preprint. April 22, 2026.

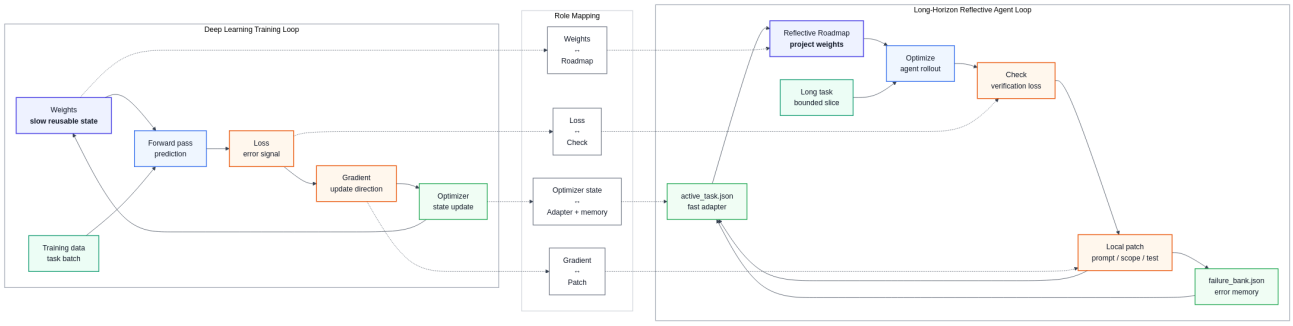

Figure 1. Training view of long-horizon agent loops. `RLS` does not update neural parameters; it imports the training structure into external state. The roadmap is the slow reusable state, while checks, patches, adapters, and failure memory implement a non-parametric update loop.

across downstream tasks, the roadmap carries project-level structure across agent epochs. Fast local state can adapt, but it should adapt against the stable backbone rather than rewriting the whole objective.

This paper makes three contributions. First, we formulate long-task coding loops through a roadmap-as-weights abstraction. Second, we describe a concrete seven-CLI implementation that keeps state, logs, and handoffs outside ephemeral chat context. Third, we show how the same abstraction scales to `moa-loop`: a mixture-of-agents computation graph with vertical serial epochs, horizontal DAG scheduling, and shared blackboard communication.

2. Preliminaries and Problem Formulation

Coding-agent episodes. Let q denote a repository task and let an agent policy π interact with a workspace through tool calls, shell commands, file edits, and tests. A one-shot agent produces a trajectory

$$\tau \sim \pi(\cdot \mid q), \tag{1}$$

where τ contains observations, actions, tool outputs, and edits. For short tasks, a single trajectory may be enough. For long-horizon engineering, however, the desired object is not one trajectory but a sequence of recoverable epochs.

Definition 2.1 (Reflective loop state)**.** A reflective loop state is a tuple

$$\mathcal{L}^{(k)} = \langle R, A^{(k)}, F^{(k)}, M^{(k)}, H^{(k)} \rangle, \tag{2}$$

where R is the roadmap, $A^{(k)}$ is the active task adapter, $F^{(k)}$ is the failure memory, $M^{(k)}$ records the current mode, and $H^{(k)}$ stores logs, snapshots, and handoff notes at epoch k.

Slow and fast state. The roadmap R stores durable project intent: goals, constraints, milestones, and non-negotiable design choices. It is analogous to a frozen or slowly updated backbone. The active adapter $A^{(k)}$ is a structured JSON record for the current slice of work. It stores local patches, immediate verification requirements, and the next bounded target. This separation prevents two common failures: global plans being overwritten by local errors, and local fixes being lost inside a long transcript.

Optimize/check epochs. Each epoch contains two conceptual phases. The optimize phase performs bounded implementation conditioned on $(R, A^{(k)}, F^{(k)})$. The check phase reviews the result, estimates the residual error, and emits patch signals:

$$P^{(k)} = C(\tau^{(k)}, R, A^{(k)}, F^{(k)}), \tag{3}$$

where $P^{(k)}$ may include prompt patches, scope patches, verification patches, and new failure patterns. The next state is updated externally:

$$A^{(k+1)}, F^{(k+1)}, H^{(k+1)} = U(A^{(k)}, F^{(k)}, H^{(k)}, P^{(k)}). \tag{4}$$

This update is non-parametric: it changes files and structured state, not model weights.

Objective. Let $S(\tau, q)$ be a task success indicator and let $B(\tau)$ measure engineering cost, such as repeated failed attempts or redundant context reconstruction. `RLS` aims to improve long-horizon execution by increasing expected success while reducing repeated work:

$$\max_{\mathcal{L}} \; \mathbb{E}_q \left[S(\tau, q) - \lambda B(\tau) \right], \tag{5}$$

subject to auditability: every epoch should leave enough state for a later agent or human to resume the project without relying on hidden context.

3. `RLS` Design

3.1. Long Tasks First

`RLS` starts from a practical observation: the hard part of autonomous coding is not one patch, but continuity across

many patches. A long task needs a stable objective, bounded local work, periodic checking, and a way to resume after the model, process, or human session stops. We therefore treat the loop as the primary unit of design. The CLI call is only one forward pass inside a larger training-like procedure.

3.2. Unified Loop Skill Interface

`RLS` packages the same long-task loop abstraction for seven agent CLIs plus one multi-agent orchestrator. Each skill directory contains a human-readable `SKILL.md`, a default prompt, an initialization script, a daemon runner, and state files. The CLI-specific layer handles dispatch and model invocation; the loop semantics remain shared. This design makes the loop portable: Codex, Gemini, Claude, Qwen, Kimi, MiniMax, and Cursor can use different command interfaces while preserving the same state machine.

3.3. Roadmaps as Weights

The central design choice is to make the deep-learning metaphor operational. Table 2 summarizes the mapping. The roadmap is not a prompt fragment; it is the durable backbone of the project. In practice, we found that roadmap quality dominates long-task stability in the same way that pretrained weights dominate downstream adaptation. `active_task.json` is not a log; it is the fast local adapter that captures the current slice. The check phase is not merely a reviewer; it computes backward signals that update the next optimize phase. `failure_bank.json` is not a transcript archive; it is an abstracted memory of error patterns and mitigations.

3.4. Vertical Iteration

The vertical loop is intentionally serial. At epoch k, the agent reads the roadmap, failure bank, and active adapter; performs one bounded optimize pass; runs or records verification; then enters a check pass. The check pass writes structured local patches and updates failure memory. Epoch $k+1$ begins only after the previous state has been persisted. This ordering trades some latency for correctness: project state remains traceable, interruptions are recoverable, and a later agent can inspect why a decision was made.

3.5. Loop Trigger Boundaries

Different systems implement "loop" at different trigger boundaries. Claude/Ralph-style loops intercept the runtime stop point: a Stop hook can return a blocking decision and refeed a reason into the next model step. Codex-style loops use external time: a shell or Python daemon sleeps, wakes, and resumes the same thread through `codex exec resume`. LikeCode-style `/loop` behaves like cron at the user level, but it is not Linux `crontab`; it is an application-level scheduler that creates tasks and later enqueues prompts inside the running app. `RLS` abstracts over these mechanisms by requiring every trigger to produce the same epoch contract: read roadmap, run a bounded slice, check, persist patches, and prepare the next epoch.

Algorithm 1 `RLS` vertical epoch

1: Read roadmap R, adapter $A^{(k)}$, and failure bank $F^{(k)}$
2: Select one bounded target from $A^{(k)}$
3: Run optimize pass and record trajectory $\tau^{(k)}$
4: Run check pass against R, target, and verification evidence
5: Emit local patches $P^{(k)}$
6: Persist updated adapter, failure bank, mode, logs, and handoff

3.6. `moa-loop`: From One Loop to a Computation Graph

The repository also includes `moa-loop`, and it should be part of the paper because it answers the scaling question: what happens after a single long-task loop becomes multiple cooperating agents? We treat `moa-loop` as the multi-agent generalization of `RLS`, not as an eighth CLI wrapper. In this view, the roadmap remains the frozen backbone, subtask adapters act like PEFT-style local updates, the optimize/check cycle remains the training epoch, and the DAG becomes the computation graph.

Concretely, independent nodes such as research, documentation, and tests may run in parallel, while dependent nodes remain serial. A centralized shared blackboard stores horizontal working state, avoiding point-to-point message storms. The key rule is separation of concerns: horizontal coordination optimizes latency inside an epoch, while vertical persistence preserves ordered project evolution across epochs. Thus `moa-loop` is the bridge from persistent single-agent loops to mixture-of-agents training dynamics.

3.7. Two-Layer Communication

The communication design follows the reference architecture in `references/ref1.txt`. Horizontal communication favors a shared blackboard for low-latency multi-agent coordination. Vertical communication favors structured persistence: Markdown for human-readable intent, JSON for compact active state, and SQL or file-backed logs for durable indexing and replay. The two layers should not replace each other. Shared memory is fast but ephemeral; persistent JSON and logs are slower but auditable and resumable.

Skill	CLI target	State directory	Initializer	Core persistent state
codex-loop	codex	.codex-loop/state	init_codex_loop.cjs	roadmap, active_task.json, failure_bank.json, last_mode.txt
gemini-loop	gemini	.gemini-loop/state	init_gemini_loop.cjs	roadmap, active_task.json, failure_bank.json, last_mode.txt
claude-loop	claude	.claude-loop/state	init_claude_loop.cjs	roadmap, active_task.json, failure_bank.json, last_mode.txt
qwen-loop	qwen	.qwen-loop/state	init_qwen_loop.cjs	roadmap, active_task.json, failure_bank.json, last_mode.txt
kimi-loop	kimi	.kimi-loop/state	init_kimi_loop.cjs	roadmap, active_task.json, failure_bank.json, last_mode.txt
minimax-loop	mmx	.minimax-loop/state	init_minimax_loop.cjs	roadmap, active_task.json, failure_bank.json, last_mode.txt
cursor-loop	cursor	.cursor-loop/state	init_cursor_loop.cjs	roadmap, active_task.json, failure_bank.json, last_mode.txt
moa-loop	multi-agent	.moa-loop/state	init_moa_loop.cjs	roadmap, blackboard, DAG, iterations

Table 1. The eight loop skills share the same persistent state abstraction while adapting to different agent CLIs. moa-loop extends the interface with DAG scheduling and shared blackboard.

ML concept	RLS component
Pretrained backbone	Roadmap / global plan
PEFT or LoRA adapter	active_task.json local state
Forward pass	Optimize phase
Loss signal	Check phase findings
Backpropagation signal	Local patches
Training epoch	One optimize/check cycle
Checkpoint	Snapshot and handoff logs
Catastrophic forgetting control	failure_bank.json

Table 2. The deep-learning analogy used by RLS is non-parametric: all updates are external state updates.

3.8. Daemon Management and Health Supervision

Every loop skill ships a unified daemon management interface. Four scripts—start, stop, status, and print_cron_entry—provide OS-level process supervision independent of the agent CLI. The start script launches the daemon in a detached tmux session or nohup background process, writes a PID file for health checks, and refuses to start a second instance. The stop script sends SIGTERM, waits for graceful shutdown, and falls back to SIGKILL if the process does not exit within five seconds. The status script reads the PID file, checks whether the process is alive, and reports the current mode plus recent log activity.

The print_cron_entry script generates ready-to-use crontab entries: an @reboot rule for auto-starting after machine restart, and a periodic health-check rule that runs the status script and triggers a restart on failure. Each skill uses a health-check interval tuned to its workload characteristics, summarized in Table 4.

This separation of concerns means that the loop optimization logic inside each skill remains independent of how the process is supervised. A developer can run the daemon manually, under tmux, via cron, or under systemd, and the epoch contract remains the same.

4. Lightweight Validation

This paper does not claim benchmark-scale empirical improvements. Instead, we validate whether the repository realizes the proposed long-task abstraction and whether the artifacts support the intended roadmap-to-adapter-to-patch workflow.

Structural validation. We inspect the eight skill directories and verify that each exposes the same interface family: SKILL.md, an initializer, a daemon runner, daemon management scripts, and persistent state files. Table 1 shows the resulting comparison. The shared structure supports the main portability claim: loop semantics are independent of the underlying CLI target.

Roadmap-to-patch case. A typical epoch starts from the roadmap and selects a bounded task in active_task.json. The optimize phase attempts the task and records evidence. If verification fails, the check phase should not simply report failure. It should produce a localized patch, for example: a prompt patch that changes how the agent reasons, a scope patch that narrows the edit radius, or a verification patch that adds a missing test. The next optimize phase consumes the patch while still being constrained by the roadmap backbone. This makes the failure reusable across epochs without requiring the next agent to reread the entire transcript.

Design-level ablation. Table 5 summarizes expected degradations when individual components are removed. These are not presented as measured benchmark results; they are acceptance criteria for the architecture. For example, if removing failure_bank.json does not make repeated failures harder to prevent, then the memory abstraction is not carrying useful information. If removing the check phase does not change future behavior, then the loop is only logging rather than optimizing.

Takeaway. The repository satisfies the core implementation requirements for long-task loop optimization: shared loop structure across CLIs, explicit roadmap/adapter state separation, auditable mode transitions, and an extension path from single-agent vertical epochs to multi-agent DAG coordination.

5. Related Work

LLM agents and coding agents. Recent LLM agents use tools, executable actions, and environment feedback to solve

Mechanism	Trigger boundary	Continuation method	State implication
Claude/Ralph Hook	Runtime `Stop` event	Block stop and refeed reason	Strong lifecycle control, runtime-specific
Codex external daemon	External `while`/sleep tick	Resume same thread with prompt	Portable, process-managed state and logs
LikeCode `/loop`	Application-level cron scheduler	Enqueue scheduled prompt	User-friendly task registry, app-managed timing
OS-level cron	System scheduler (`crontab`)	Health-check and restart daemon	Machine-level supervision, reboots, crash recovery
`RLS` contract	Persistent epoch boundary	Optimize/check with roadmap state	Trigger-agnostic long-task optimization

Table 3. Loop mechanisms differ in where they trigger continuation. `RLS` focuses on the state contract that should survive any trigger boundary.

Skill	Interval	Rationale
`gemini-loop`	`*/5`	Fast iteration, reference agent
`codex-loop`	`*/10`	Balanced, production-stable
`claude-loop`	`*/10`	Production-grade stability
`cursor-loop`	`*/15`	IDE integration, lower frequency
`kimi-loop`	`*/15`	Long-context tasks, wider interval
`minimax-loop`	`*/5`	Lightweight, fast patrol
`qwen-loop`	`*/10`	Balanced strategy
`moa-loop`	`*/5`	Multi-agent, frequent checks

Table 4. Differential cron health-check intervals. Faster intervals target lightweight or multi-agent skills; slower intervals suit IDE-integrated or long-context skills.

Removed component	Expected degradation	Observable symptom
Roadmap	Plan drift	Local fixes violate global goals
`active_task.json`	Weak adaptation	Next epoch repeats setup work
`failure_bank.json`	Forgetting	Recurring errors reappear
Check phase	No backward signal	Failures become pass/fail notes
Logs/snapshots	Poor auditability	Resume depends on chat context
DAG scheduler	Low parallelism	Independent work is serialized
Cron health check	No crash recovery	Daemon stays dead after failure

Table 5. Lightweight ablation analysis of the reflective loop design.

increasingly complex tasks (Wang et al., 2024b; Novikov et al., 2025; Ou et al., 2025). These systems demonstrate that agentic execution can outperform static prompting when tasks require interaction and revision. `RLS` is complementary: it does not propose a new base policy, but a persistence layer that helps coding agents continue work across repeated sessions and checks.

Experience learning and agent memory. Prior work studies experiential learning, workflow memory, procedural memory, and agentic memory (Zhao et al., 2024; Wang et al., 2025; Fang et al., 2025; Han et al., 2025; Xu et al., 2025; Cao et al., 2025). These methods store reusable knowledge from prior trajectories. `RLS` shares the goal of reuse, but targets repository-level engineering state rather than only task-level behavior. Its memory is split into roadmap, active adapter, logs, and failure bank so that different temporal scales can evolve independently.

Skills and reusable procedures. Skill libraries provide reusable abstractions for agent behavior, ranging from embodied skills to tool-use procedures (Wang et al., 2024a; Anthropic, 2025). `RLS` treats each loop skill as a reusable orchestration skill: it teaches a CLI agent how to plan, optimize, check, and persist state. The focus is therefore not a domain skill such as web navigation, but a meta-skill for sustained engineering.

Multi-agent orchestration. Multi-agent systems often require dependency-aware scheduling and shared communication substrates. The horizontal DAG extension in `RLS` follows this direction by separating parallelizable subtasks from causally dependent ones. The vertical loop ensures that such parallelism does not erase temporal order: every epoch still produces a recoverable state transition.

Loop mechanisms in agent runtimes. Existing agent products expose different continuation mechanisms. Hook-based systems can intercept lifecycle events such as stopping; daemon-based systems can resume an external process on a timer; and application-level cron systems can schedule prompts inside the app event loop. These mechanisms differ operationally, but they share the same research question for long tasks: what state must persist between invocations? `RLS` argues that the roadmap, adapter, patches, and failure bank are the minimum state needed to make those invocations behave like optimization rather than repetition.

6. Conclusion

We presented `Reflective Loop Skills`, a systems pattern for turning long-horizon coding-agent work into persistent, auditable optimization loops. The core lesson is simple: for long tasks, roadmaps are weights. They carry the slow project knowledge that local adapters and patches should update against, not overwrite. Across seven CLI-specific skills and one multi-agent orchestrator, `RLS` uses the same non-parametric deep-learning analogy: roadmaps act as slow backbones, active task files act as fast adapters, check phases produce backward signals, local patches guide the next optimize pass, and failure banks preserve reusable error memory. We further described hook, daemon, OS-level cron, and application-level loop boundaries, and included `moa-loop` as the natural extension from one persistent loop to a mixture-of-agents computation graph.

The immediate value of `RLS` is practical: it gives coding agents a concrete way to continue long-running work without relying on hidden chat context. Future work should add

benchmark-scale evaluation, stronger state schemas, richer visual monitoring, and adapters for hosted agent runtimes where file-system persistence is not available by default.

Impact Statement

This work aims to make long-running coding agents more transparent, reproducible, and recoverable. Persistent roadmaps and failure memories may reduce repeated computation and make agentic engineering easier to audit. The same persistence can also preserve stale assumptions or unsafe behaviors if the check phase is weak; therefore, RLS treats the roadmap and failure memory as inspected artifacts rather than hidden model state.

Limitations

RLS is a systems pattern and repository implementation, not a trained model. The present paper reports lightweight structural validation rather than benchmark-scale task success rates. The phrase "roadmaps as weights" is a conceptual analogy for non-parametric state updates; it should not be read as gradient training of neural parameters. The current implementation also assumes file-system persistence and CLI agents, so hosted or stateless agent runtimes require adapters.

References

Anthropic. Skills. https://github.com/anthropics/skills, 2025. GitHub repository.

Cao, Z., Deng, J., Yu, L., Zhou, W., Liu, Z., Ding, B., and Zhao, H. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution, 2025. URL https://arxiv.org/abs/2512.10696.

Fang, R., Liang, Y., Wang, X., Wu, J., Qiao, S., Xie, P., Huang, F., Chen, H., and Zhang, N. Memp: Exploring agent procedural memory. *CoRR*, abs/2508.06433, 2025. doi: 10.48550/ARXIV.2508.06433. URL https://doi.org/10.48550/arXiv.2508.06433.

Han, D., Couturier, C., Díaz, D. M., Zhang, X., Rühle, V., and Rajmohan, S. LEGOMem: Modular procedural memory for multi-agent LLM systems for workflow automation. *CoRR*, abs/2510.04851, 2025. doi: 10.48550/ARXIV.2510.04851. URL https://doi.org/10.48550/arXiv.2510.04851.

Novikov, A., Vu, N., Eisenberger, M., Dupont, E., Huang, P., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., Kumar, M. P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., and Balog, M. Alphaevolve: A coding agent for scientific and algorithmic discovery. *CoRR*, abs/2506.13131, 2025. doi: 10.48550/ARXIV.2506.13131. URL https://doi.org/10.48550/arXiv.2506.13131.

Ou, Y., Luo, Y., Zheng, J., Wei, L., Qiao, S., Zhang, J., Zheng, D., Chen, H., and Zhang, N. Automind: Adaptive knowledgeable agent for automated data science. *CoRR*, abs/2506.10974, 2025. doi: 10.48550/ARXIV.2506.10974. URL https://doi.org/10.48550/arXiv.2506.10974.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a. URL https://openreview.net/forum?id=ehfRiF0R3a.

Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H. Executable code actions elicit better LLM agents. In *Forty-first International Conference on Machine Learning*. OpenReview.net, 2024b. URL https://openreview.net/forum?id=jJ9BoXAfFa.

Wang, Z. Z., Mao, J., Fried, D., and Neubig, G. Agent workflow memory. In *Forty-second International Conference on Machine Learning*. OpenReview.net, 2025. URL https://openreview.net/forum?id=NTAhi2JEEE.

Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. A-MEM: Agentic memory for LLM agents, 2025. URL https://arxiv.org/abs/2502.12110.

Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y., and Huang, G. Expel: LLM agents are experiential learners. In *Thirty-Eighth AAAI Conference on Artificial Intelligence*, pp. 19632–19642. AAAI Press, 2024. doi: 10.1609/AAAI.V38I17.29936. URL https://doi.org/10.1609/aaai.v38i17.29936.

A. Appendix

A.1. Repository Layout

Each primary loop skill follows the same layout:

```
<name>-loop/
  README.md
  SKILL.md
  prompt.md
  scripts/
    init_<name>_loop.cjs
    run_daemon.py
    run_daemon.sh
    start_<name>_loop.sh
    stop_<name>_loop.sh
    status_<name>_loop.sh
    print_cron_entry.sh
```

A.2. Example Active Task Schema

```
{
  "plan_id": "PROJECT_ROADMAP",
  "mode": "optimize",
  "bounded_target": "complete one verifiable slice",
  "prompt_patch": [],
  "scope_patch": [],
  "verification_patch": [],
  "handoff": "next smallest useful action"
}
```

A.3. Example Failure Memory Entry

```
{
  "failure_type": "verification_gap",
  "symptom": "implementation completed without running target tests",
  "patch": "before final handoff, run the smallest relevant check",
  "seen_in": ["epoch-004", "epoch-009"]
}
```

A.4. Daemon Management Commands

All loop skills share the same management interface:

```
# Initialize state
node <skill>-loop/scripts/init_<name>_loop.cjs MY_ROADMAP

# Start daemon (tmux/nohup auto-select)
bash <skill>-loop/scripts/start_<name>_loop.sh MY_ROADMAP

# Check status (PID health + last mode)
bash <skill>-loop/scripts/status_<name>_loop.sh MY_ROADMAP

# Stop daemon gracefully (SIGTERM -> SIGKILL)
bash <skill>-loop/scripts/stop_<name>_loop.sh MY_ROADMAP
```

```
# Print cron entries for OS-level supervision
bash <skill>-loop/scripts/print_cron_entry.sh
```

A.5. Cron Integration Example

The following crontab entries provide automatic restart after reboot and periodic health checks:

```
# Auto-start after reboot
@reboot cd /workspace && <SKILL>_LOOP_WORKSPACE=/workspace \
  bash /path/to/start_<name>_loop.sh

# Health check every N minutes (restart on failure)
*/N * * * * cd /workspace && \
  bash /path/to/status_<name>_loop.sh \
  || bash /path/to/start_<name>_loop.sh
```

A.6. Implementation Notes

The Codex and MOA loops include additional monitoring commands:

```
# Codex: real-time monitoring dashboard
bash codex-loop/scripts/monitor_codex_loop.sh --watch

# MOA: DAG + Blackboard + PEFT dashboard
bash moa-loop/scripts/monitor_moa_loop.sh --watch
```

The MOA loop extends the abstraction with `dag_scheduler.py`, `shared_blackboard.py`, and `iteration_manager.py`. These modules implement horizontal DAG scheduling, centralized agent communication, and epoch-level PEFT adaptation tracking.